

```

"""
metricas.py
=====

Biblioteca de METRICAS estadísticas del pipeline. Es código puro de cálculo:
no toma decisiones de selección ni escribe archivos. Las fases lo consumen.

Contenido
-----
* Discretización supervisada por cuantiles (binning) para WOE/IV.
* Weight of Evidence (WOE) e Information Value (IV).
* Gini (a partir del AUC ROC) para target binario y continuo.
* Medidas de asociación mixtas: Spearman/Pearson, V de Cramer y razón de
  correlación (eta) -> matriz única comparable en [0, 1].
* Factor de inflación de la varianza (VIF).
* Population Stability Index (PSI) para estabilidad temporal del panel.

Convenciones
-----
* ``evento`` = target == 1 (el suceso que se quiere predecir, p. ej. default).
* ``no evento`` = target == 0.
* Todas las funciones devuelven ``np.nan`` (no lanzan) cuando la métrica no es
  calculable, y la fase que las llama documenta el motivo. Nunca se elimina una
  variable "en silencio" por un fallo de cálculo.
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd

from .logging_utils import obtener_logger

LOGGER = obtener_logger("metricas")

#: Etiqueta reservada para el bin de valores ausentes. Los nulos NO se imputan:
#: la ausencia suele ser informativa (no reportar un dato es una señal), así que
#: forma su propio bin y participa del IV con su propio WOE.
BIN_NULOS = "__NULOS__"
#: Etiqueta para categorías residuales agrupadas por baja frecuencia.
CAT_OTROS = "__OTROS__"
#: Número de categorías más frecuentes que NUNCA se agrupan en __OTROS__.
#: Garantiza que una variable de alta cardinalidad conserve al menos algunos
#: niveles propios y no se convierta en un único bin (IV = 0 artificial).
MIN_NIVELES_CATEGORICOS = 8

# =====
# 1. DISCRETIZACION (BINNING)
# =====
def binear_numerica(
    x: pd.Series, n_bins: int = 10, min_prop_bin: float = 0.03
) -> tuple[pd.Series, str]:
    """Discretiza una variable numérica en bins de igual frecuencia (cuantiles).

    Por que cuantiles y no intervalos de igual ancho: los intervalos de igual
    ancho colapsan cuando la distribución es asimétrica (típico en montos,
    saldos o conteos), dejando bins vacíos que inflan artificialmente el IV.
    Los cuantiles garantizan masa comparable por bin y por lo tanto un WOE
    estimado con error similar en cada tramo.

    El número de bins se reduce automáticamente hasta que cada bin tenga al
    menos ``min_prop_bin`` de la muestra, porque un WOE calculado sobre 5
    observaciones es ruido, no señal.

    Returns
    -----
    (bins, metodo)
    ``bins`` es una serie de etiquetas de texto (incluye ``__NULOS__``);

```

```

    ``metodo`` describe en texto lo que se hizo, para la bitacora.
"""
x = pd.to_numeric(x, errors="coerce")
no_nulos = x.dropna()

if no_nulos.empty:
    return pd.Series([BIN_NULOS] * len(x), index=x.index), "sin_datos_no_nulos"

n_unicos = int(no_nulos.nunique())

# Pocos valores distintos -> cada valor es su propio bin (no tiene sentido
# cuantilizar una variable que solo toma 3 valores).
if n_unicos <= max(2, min(n_bins, 20)) and n_unicos <= 20:
    etiquetas = x.astype("object").where(x.notna(), BIN_NULOS).astype(str)
    return etiquetas, f"valores_discretos ({n_unicos} niveles)"

# Se busca el mayor numero de bins que respete el tamano minimo por bin.
bins_max = max(2, min(n_bins, int(np.floor(1.0 / min_prop_bin))))
for k in range(bins_max, 1, -1):
    try:
        cortes = pd.qcut(no_nulos, q=k, duplicates="drop", retbins=True)[1]
    except (ValueError, IndexError):
        continue
    if len(cortes) < 3: # menos de 2 bins efectivos
        continue
    cortes = np.unique(cortes)
    cortes[0], cortes[-1] = -np.inf, np.inf
    cat = pd.cut(x, bins=cortes, include_lowest=True)
    conteo = cat.value_counts(normalize=True, dropna=True)
    if conteo.min() >= min_prop_bin * 0.9: # 10% de tolerancia
        etiquetas = cat.astype("object")
        etiquetas = pd.Series(
            np.where(x.isna(), BIN_NULOS, etiquetas.astype(str)), index=x.index
        )
        return etiquetas, f"cuantiles q={len(cortes)-1} (min_prop_bin={min_prop_bin})"

# Fallback: mediana como unico corte. Si ni eso, un solo bin (IV = 0).
mediana = float(no_nulos.median())
etiquetas = pd.Series(
    np.where(x.isna(), BIN_NULOS, np.where(x <= mediana, "<=mediana", ">mediana")),
    index=x.index,
)
if etiquetas[x.notna()].nunique() < 2:
    return pd.Series([BIN_NULOS if v else "unico" for v in x.isna()], index=x.index), "bin_unico"
return etiquetas, "corte_por_mediana (distribucion muy concentrada)"

def binear_categorica(
    x: pd.Series, max_categorias: int = 50, min_prop_bin: float = 0.0
) -> tuple[pd.Series, str]:
    """Prepara una variable categorica para el calculo de WOE/IV.

    Tratamiento aplicado:
    1. Los nulos pasan a la categoria explicita ``__NULOS__``.
    2. Si la cardinalidad supera ``max_categorias``, las categorias menos
       frecuentes se agrupan en ``__OTROS__`` conservando las mas frecuentes.
       Esto evita el sobreajuste tipico de las variables de alta cardinalidad
       (un ID disfrazado de categoria alcanzaria IV "perfecto" y seria inutil
       fuera de muestra).
    3. Categorias con frecuencia menor a ``min_prop_bin`` tambien van a
       ``__OTROS__`` por inestabilidad del WOE.
    """
    s = x.astype("object").where(x.notna(), BIN_NULOS).astype(str)
    frec = s.value_counts(normalize=True)
    n_cat = len(frec)
    acciones: list[str] = []

    # Se PROTEGE siempre un nucleo de las categorias mas frecuentes. Sin esta
    # garantia, una variable de cardinalidad muy alta (180 sucursales al 0.5%
    # cada una) veria TODAS sus categorias por debajo de min_prop_bin y se
    # colapsaria en un unico bin, con IV = 0 por construccion: la variable

```

```

# quedaria descartada por un artefacto del binning y no por falta de senal.
n_protegidas = max(MIN_NIVELES_CATEGORICOS, 0)
protegidas = set(frec.head(min(n_protegidas, max_categorias)).index) | {BIN_NULOS}

raras = set()
if min_prop_bin > 0:
    raras |= set(frec[frec < min_prop_bin].index)
if n_cat > max_categorias:
    raras |= set(frec.index) - set(frec.head(max_categorias).index)
raras -= protegidas

if raras:
    s = s.where(~s.isin(raras), CAT_OTROS)
    acciones.append(f"{len(raras)} categorias agrupadas en {CAT_OTROS}")

metodo = f"categorias_directas ({n_cat} niveles originales)"
if acciones:
    metodo += "; " + "; ".join(acciones)
return s, metodo

# =====
# 2. WEIGHT OF EVIDENCE E INFORMATION VALUE
# =====
@dataclass
class ResultadoIV:
    """Salida del calculo de IV, con la tabla WOE completa para auditoria."""

    iv: float
    tabla: pd.DataFrame
    mapa_woe: dict[str, float]
    n_bins_efectivos: int
    metodo: str
    observacion: str = ""

def tabla_woe(
    bins: pd.Series, y: np.ndarray, correccion: float = 0.5
) -> tuple[pd.DataFrame, float]:
    """Construye la tabla WOE y calcula el Information Value.

Definiciones
-----
Para cada bin *i*:

    dist_evento_i    = eventos en i    / total de eventos
    dist_no_evento_i = no eventos en i / total de no eventos

    WOE_i = ln( dist_no_evento_i / dist_evento_i )
    IV_i  = (dist_no_evento_i - dist_evento_i) * WOE_i
    IV    = suma de IV_i

El WOE mide, en escala logaritmica, cuanto se desvia un bin de la mezcla
poblacional: 0 significa que el bin tiene la misma composicion que el
total (no aporta informacion). El IV pondera esa desviacion por la masa
que la respalda, de modo que un bin muy discriminante pero casi vacio
contribuye poco. Por eso el IV es una medida de *poder predictivo
global* y no solo de separacion local.

Correccion de continuidad (Haldane-Anscombe): se suma ``correccion``
(0.5 por defecto) a los conteos de cada bin. Sin ella, un bin sin eventos
produce ln(0) = -inf y un IV infinito. Con ella el estimador queda
ligeramente sesgado hacia cero, que es el sesgo conservador deseable.

Escala de referencia (Siddiqi, *Credit Risk Scorecards*):

    IV < 0.02      sin poder predictivo
    0.02 - 0.10    poder debil
    0.10 - 0.30    poder medio
    0.30 - 0.50    poder fuerte
    > 0.50         sospechoso: revisar posible fuga de informacion

```

```

"""
df = pd.DataFrame({"bin": bins.astype(str).values, "y": np.asarray(y, dtype=float)})
df = df[~df["y"].isnull()]

agrupado = df.groupby("bin", observed=True)["y"].agg(n="size", eventos="sum")
agrupado["no_eventos"] = agrupado["n"] - agrupado["eventos"]

total_ev = float(agrupado["eventos"].sum())
total_no = float(agrupado["no_eventos"].sum())
if total_ev <= 0 or total_no <= 0:
    agrupado["woe"] = 0.0
    agrupado["iv_bin"] = 0.0
    return agrupado.reset_index(), 0.0

k = len(agrupado)
# Correccion aplicada tambien a los totales para que las distribuciones sumen 1.
agrupado["dist_evento"] = (agrupado["eventos"] + correccion) / (total_ev + correccion * k)
agrupado["dist_no_evento"] = (agrupado["no_eventos"] + correccion) / (total_no + correccion * k)

agrupado["woe"] = np.log(agrupado["dist_no_evento"] / agrupado["dist_evento"])
agrupado["iv_bin"] = (agrupado["dist_no_evento"] - agrupado["dist_evento"]) * agrupado["woe"]
agrupado["pct_n"] = agrupado["n"] / agrupado["n"].sum()
agrupado["tasa_evento"] = agrupado["eventos"] / agrupado["n"].replace(0, np.nan)

iv = float(agrupado["iv_bin"].sum())
return agrupado.reset_index(), iv

def calcular_iv(
    x: pd.Series,
    y: pd.Series,
    tipo: str,
    n_bins: int = 10,
    min_prop_bin: float = 0.03,
    max_categorias: int = 50,
    correccion: float = 0.5,
) -> ResultadoIV:
    """Calcula el IV de una variable frente a un target binario.

    Selecciona automaticamente el tratamiento segun el tipo de la variable y
    documenta cual uso (requisito de trazabilidad del proyecto).
    """
    if tipo in ("NUMERICA", "BOOLEANA"):
        bins, metodo = binear_numerica(x, n_bins=n_bins, min_prop_bin=min_prop_bin)
    else:
        bins, metodo = binear_categorica(x, max_categorias=max_categorias, min_prop_bin=min_prop_bin)

    tabla, iv = tabla_woe(bins, y.values, correccion=correccion)
    mapa = dict(zip(tabla["bin"].astype(str), tabla["woe"].astype(float)))

    obs = ""
    n_efectivos = int(tabla["bin"].nunique())
    if n_efectivos < 2:
        obs = "Un solo bin efectivo: la variable no separa la muestra; IV = 0 por construccion."
    return ResultadoIV(iv=iv, tabla=tabla, mapa_woe=mapa, n_bins_efectivos=n_efectivos,
                       metodo=metodo, observacion=obs)

def aplicar_woe(bins: pd.Series, mapa_woe: dict[str, float]) -> pd.Series:
    """Proyecta una variable bineada al espacio WOE.

    Sirve para calcular el Gini de variables categoricas y de variables
    numericas con relacion no monotona con el target: una vez transformada a
    WOE, la variable queda ordenada por riesgo y el AUC es interpretable.
    """
    return bins.astype(str).map(mapa_woe).astype(float).fillna(0.0)

# =====
# 3. GINI / AUC
# =====

```

```

def gini_desde_auc(auc: float) -> float:
    """Convierte AUC a coeficiente de Gini: ``Gini = 2*AUC - 1``.

    Interpretacion: AUC 0.5 (moneda al aire) -> Gini 0. AUC 1 -> Gini 1.
    El Gini es la razon entre el area que la curva de Lorenz del modelo gana
    sobre la diagonal y el area maxima teorica alcanzable.
    """
    return float(2.0 * auc - 1.0)

def calcular_gini(
    puntaje: pd.Series, y: pd.Series, usar_absoluto: bool = True
) -> tuple[float, float]:
    """Gini de una variable continua frente a un target binario.

    Parameters
    -----
    puntaje
        Variable ordinal (valores crudos o su transformacion WOE).
    y
        Target binario 0/1.
    usar_absoluto
        Si ``True`` devuelve |Gini|. En seleccion de variables interesa la
        MAGNITUD de la discriminacion, no su direccion: una variable con
        Gini -0.45 discrimina exactamente igual que una con +0.45, solo que
        con el signo invertido. El signo se conserva aparte para el analisis
        de negocio (permite detectar relaciones contraintuitivas).

    Returns
    -----
    (gini, gini_con_signo)
    """
    from sklearn.metrics import roc_auc_score

    s = pd.to_numeric(puntaje, errors="coerce")
    yy = pd.to_numeric(y, errors="coerce")
    valido = s.notna() & yy.notna()
    if valido.sum() < 10 or yy[valido].nunique() < 2:
        return np.nan, np.nan

    try:
        auc = roc_auc_score(yy[valido].astype(int), s[valido])
    except ValueError:
        return np.nan, np.nan

    g = gini_desde_auc(float(auc))
    return (abs(g) if usar_absoluto else g), g

def gini_continuo(x: pd.Series, y: pd.Series) -> tuple[float, float]:
    """Aproximacion del Gini cuando el target es CONTINUO.

    Se usa la correlacion de rangos de Spearman como sustituto: es una medida
    de concordancia monotona en [-1, 1], la misma escala del Gini, y coincide
    con el D de Somers (la generalizacion natural del Gini) salvo por el
    tratamiento de empates. Se documenta explicitamente porque no es el Gini
    clasico de clasificacion.
    """
    from scipy.stats import spearmanr

    xx = pd.to_numeric(x, errors="coerce")
    yy = pd.to_numeric(y, errors="coerce")
    valido = xx.notna() & yy.notna()
    if valido.sum() < 10:
        return np.nan, np.nan

    try:
        rho = float(spearmanr(xx[valido], yy[valido]).statistic)
    except Exception: # noqa: BLE001
        return np.nan, np.nan

    if not np.isfinite(rho):
        return np.nan, np.nan

```

```
return abs(rho), rho
```

```
# =====
# 4. ASOCIACION ENTRE VARIABLES (para la fase multivariada)
# =====
def v_de_cramer(a: pd.Series, b: pd.Series, correccion_sesgo: bool = True) -> float:
    """V de Cramer entre dos variables categoricas.

    Se deriva del estadistico chi-cuadrado de independencia y se normaliza al
    intervalo [0, 1], de modo que es comparable con un |Spearman|.

    
$$V = \sqrt{\chi^2 / (n * \min(\text{filas}-1, \text{columnas}-1))}$$


    Con ``correccion_sesgo`` se aplica la correccion de Bergsma, necesaria
    porque la V cruda esta sesgada al alza cuando hay muchas categorias y
    pocas observaciones (llegaria a declarar redundantes dos variables de alta
    cardinalidad sin relacion real).
    """
    from scipy.stats import chi2_contingency

    tabla = pd.crosstab(a.astype(str), b.astype(str))
    if tabla.shape[0] < 2 or tabla.shape[1] < 2:
        return np.nan
    try:
        chi2 = float(chi2_contingency(tabla, correction=False)[0])
    except Exception: # noqa: BLE001
        return np.nan

    n = float(tabla.to_numpy().sum())
    if n <= 0:
        return np.nan
    r, k = tabla.shape
    phi2 = chi2 / n

    if correccion_sesgo:
        phi2 = max(0.0, phi2 - (k - 1) * (r - 1) / max(n - 1, 1))
        k = k - (k - 1) ** 2 / max(n - 1, 1)
        r = r - (r - 1) ** 2 / max(n - 1, 1)

    denom = min(k - 1, r - 1)
    if denom <= 0:
        return np.nan
    return float(np.sqrt(phi2 / denom))

def razon_correlacion(x_num: pd.Series, x_cat: pd.Series) -> float:
    """Razon de correlacion eta entre una numerica y una categorica.

    
$$\eta^2 = \text{varianza explicada entre categorias} / \text{varianza total}$$


    Es el  $R^2$  de un ANOVA de un factor y vive en [0, 1]. Responde a: "cuanto
    de la dispersion de la variable numerica se explica por saber a que
    categoria pertenece la observacion". Sirve para detectar que una dummy y
    su variable numerica de origen son la misma informacion.
    """
    xn = pd.to_numeric(x_num, errors="coerce")
    valido = xn.notna() & x_cat.notna()
    if valido.sum() < 10:
        return np.nan

    xn, xc = xn[valido], x_cat[valido].astype(str)
    media_global = xn.mean()
    ss_total = float(((xn - media_global) ** 2).sum())
    if ss_total <= 0:
        return np.nan

    grupos = xn.groupby(xc, observed=True)
    ss_entre = float((grupos.count() * (grupos.mean() - media_global) ** 2).sum())
    return float(np.sqrt(max(0.0, min(1.0, ss_entre / ss_total))))
```

```

def matriz_asociacion(
    df: pd.DataFrame, tipos: dict[str, str], metodo_numerico: str = "spearman"
) -> pd.DataFrame:
    """Matriz simetrica de asociacion en [0, 1] para variables de tipo mixto.

    Combina tres medidas segun el par de tipos, todas normalizadas a [0, 1]
    para que un unico umbral (0.90) sea aplicable de forma homogenea:

    =====
    Par de variables      Medida
    =====
    numerica-numerica      |Spearman| (por defecto) o |Pearson|
    categorica-categ.      V de Cramer con correccion de sesgo
    numerica-categorica    Razon de correlacion (eta)
    =====

    Spearman es el default en lugar de Pearson porque es invariante a
    transformaciones monotonas y resistente a outliers, algo habitual en
    variables economicas del panel; Pearson solo capta relacion lineal y se
    distorsiona con colas pesadas.
    """
    columnas = list(df.columns)
    n = len(columnas)
    M = pd.DataFrame(np.eye(n), index=columnas, columns=columnas, dtype=float)

    numericas = [c for c in columnas if tipos.get(c) in ("NUMERICA", "BOOLEANA")]

    # Bloque numerico-numerico vectorizado (mucho mas rapido que par a par).
    if len(numericas) >= 2:
        corr = df[numericas].apply(pd.to_numeric, errors="coerce").corr(method=metodo_numerico).abs()
        M.loc[numericas, numericas] = corr.values

    # Bloques que involucran categoricas: par a par.
    categoricas = [c for c in columnas if c not in numericas]
    for i, a in enumerate(columnas):
        for b in columnas[i + 1:]:
            if a in numericas and b in numericas:
                continue
            if a in categoricas and b in categoricas:
                valor = v_de_cramer(df[a], df[b])
            elif a in numericas:
                valor = razon_correlacion(df[a], df[b])
            else:
                valor = razon_correlacion(df[b], df[a])
            valor = float(valor) if valor is not None and np.isfinite(valor) else 0.0
            M.loc[a, b] = M.loc[b, a] = valor

    return M.fillna(0.0)

def calcular_vif(df_num: pd.DataFrame) -> pd.Series:
    """Factor de inflacion de la varianza para un bloque de variables numericas.

    
$$VIF_j = 1 / (1 - R^2_j)$$


    donde  $R^2_j$  es el coeficiente de determinacion de regresar la variable *j*
    contra todas las demas. Se calcula de forma eficiente como el elemento
    diagonal *j* de la inversa de la matriz de correlacion.

    El VIF complementa la correlacion por pares: detecta colinealidad
    MULTIPLE, es decir, una variable que es combinacion lineal de varias
    otras sin estar fuertemente correlacionada con ninguna en particular.
    Regla practica:  $VIF > 10$  indica multicolinealidad severa.

    Se usa la pseudo-inversa porque, con variables casi redundantes, la matriz
    de correlacion es singular y la inversa exacta no existe.
    """
    X = df_num.apply(pd.to_numeric, errors="coerce")
    X = X.loc[:, X.std(numeric_only=True) > 0].dropna()
    if X.shape[1] < 2 or X.shape[0] < X.shape[1] + 2:

```

```

    return pd.Series(dtype=float)

R = X.corr(method="pearson").to_numpy()
try:
    R_inv = np.linalg.pinv(R)
except np.linalg.LinAlgError:
    return pd.Series(dtype=float)

vifs = np.diag(R_inv).astype(float)
vifs = np.where(np.isfinite(vifs) & (vifs > 0), vifs, np.nan)
return pd.Series(vifs, index=X.columns)

# =====
# 5. ESTABILIDAD TEMPORAL (especifico de panel)
# =====
def calcular_psi(
    bins_base: pd.Series, bins_comparacion: pd.Series, epsilon: float = 1e-6
) -> float:
    """Population Stability Index entre dos periodos del panel.

    PSI = suma_i (p_i - q_i) * ln(p_i / q_i)

    donde p_i y q_i son las proporciones del bin *i* en la muestra base y en
    la de comparacion. Es una divergencia de Jeffreys (KL simetrizada).

    Escala de referencia habitual en riesgo:

    PSI < 0.10    poblacion estable
    0.10 - 0.25   cambio moderado, vigilar
    PSI > 0.25    cambio poblacional severo

    En un panel esto es esencial: una variable con IV alto en la muestra
    agrupada puede deber ese poder a un unico periodo atipico. Si su
    distribucion se desplaza en el tiempo, el modelo entrenado con ella se
    degradara en produccion.
    """
    p = bins_base.astype(str).value_counts(normalize=True)
    q = bins_comparacion.astype(str).value_counts(normalize=True)
    categorias = p.index.union(q.index)
    p = p.reindex(categorias, fill_value=0.0) + epsilon
    q = q.reindex(categorias, fill_value=0.0) + epsilon
    p, q = p / p.sum(), q / q.sum()
    return float(((p - q) * np.log(p / q)).sum())

def estabilidad_temporal(
    bins: pd.Series, y: pd.Series, tiempo: pd.Series, correccion: float = 0.5
) -> dict[str, float]:
    """Perfil de estabilidad de una variable a lo largo de los periodos.

    Con los MISMOs bins globales (no se re-binea por periodo: hacerlo haria
    incomparables los tramos) se calcula:

    - IV dentro de cada periodo -> media, minimo, maximo y desviacion.
    - Coeficiente de variacion del IV: dispersion relativa del poder
      predictivo. Un CV alto significa que la variable "funciona" solo a
      ratos.
    - PSI maximo del primer periodo contra cada uno de los siguientes.
    """
    resultado = {
        "iv_medio_periodo": np.nan, "iv_min_periodo": np.nan, "iv_max_periodo": np.nan,
        "iv_std_periodo": np.nan, "iv_cv_periodo": np.nan, "psi_max": np.nan,
        "n_periodos_evaluados": 0,
    }
    periodos = sorted(pd.Series(tiempo).dropna().unique())
    if len(periodos) < 2:
        return resultado

    ivs: list[float] = []
    psis: list[float] = []

```

```

base_bins: pd.Series | None = None

for p in periodos:
    mascara = (tiempo == p).to_numpy()
    if mascara.sum() < 30: # muestra insuficiente: el IV seria ruido
        continue
    bin_p, y_p = bins[mascara], y[mascara]
    if pd.to_numeric(y_p, errors="coerce").dropna().nunique() < 2:
        continue
    _, iv_p = tabla_woe(bin_p, y_p.values, correccion=correccion)
    ivs.append(iv_p)
    if base_bins is None:
        base_bins = bin_p
    else:
        psis.append(calcular_psi(base_bins, bin_p))

if ivs:
    arr = np.array(ivs, dtype=float)
    resultado.update(
        iv_medio_periodo=float(arr.mean()),
        iv_min_periodo=float(arr.min()),
        iv_max_periodo=float(arr.max()),
        iv_std_periodo=float(arr.std(ddof=0)),
        iv_cv_periodo=float(arr.std(ddof=0) / arr.mean()) if arr.mean() > 1e-12 else np.nan,
        n_periodos_evaluados=len(arr),
    )
if psis:
    resultado["psi_max"] = float(np.max(psis))
return resultado

# =====
# 6. NORMALIZACION Y SCORE COMPUESTO
# =====
def normalizar(serie: pd.Series, metodo: str = "minmax") -> pd.Series:
    """Lleva una metrica al intervalo [0, 1] para poder ponderarla.

    ``minmax``
        (x - min) / (max - min). Conserva la distancia relativa entre
        variables, pero es sensible a un outlier de IV muy alto, que comprime
        al resto hacia cero.
    ``rank``
        Percentil del rango. Ignora la magnitud y conserva solo el orden:
        robusto ante outliers, a costa de perder la nocion de "cuanto mejor".

    IV y Gini no son comparables en crudo: el Gini esta acotado en [0, 1]
    mientras que el IV no tiene cota superior. Sumarlos sin normalizar dejaria
    que el IV dominara mecanicamente la decision.
    """
    s = pd.to_numeric(serie, errors="coerce")
    if s.notna().sum() == 0:
        return pd.Series(np.nan, index=serie.index)

    if metodo == "rank":
        return s.rank(pct=True, na_option="keep")

    minimo, maximo = float(s.min()), float(s.max())
    if not np.isfinite(minimo) or not np.isfinite(maximo) or maximo - minimo < 1e-12:
        # Todas iguales: ninguna destaca, se les asigna el punto medio.
        return pd.Series(np.where(s.notna(), 0.5, np.nan), index=serie.index)
    return (s - minimo) / (maximo - minimo)

def score_compuesto(
    gini: pd.Series, iv: pd.Series, peso_gini: float = 0.5,
    peso_iv: float = 0.5, metodo: str = "minmax",
) -> pd.DataFrame:
    """Combina Gini e IV en un unico indicador de poder predictivo.

    score = peso_gini * gini_normalizado + peso_iv * iv_normalizado

```

Justificacion de usar ambas metricas y no una sola:

- El **Gini** mide capacidad de ORDENAMIENTO global (¿los casos con evento reciben puntajes mas altos?). Es insensible a donde se concentra la separacion y penaliza las relaciones no monotonas.
- El **IV** mide la informacion acumulada BIN A BIN, por lo que capta relaciones no monotonas (en U, por tramos) que el Gini crudo pierde, pero puede inflarse con bins de poca masa.

Son complementarias: una variable fuerte en ambas es una apuesta segura; una fuerte en solo una merece revision manual. Ponderar 50/50 evita privilegiar por defecto una vision sobre la otra, y los pesos quedan parametrizados para escenarios donde el negocio prefiera una de las dos.

Nota de interpretacion: el score es RELATIVO al conjunto de variables evaluado, porque la normalizacion usa el minimo y el maximo observados. No es comparable entre corridas con universos de variables distintos.

```
"""
gini_n = normalizar(gini, metodo)
iv_n = normalizar(iv, metodo)
total = peso_gini + peso_iv
pg, pi = peso_gini / total, peso_iv / total

return pd.DataFrame(
    {
        "gini_normalizado": gini_n,
        "iv_normalizado": iv_n,
        "score_compuesto": (pg * gini_n.fillna(0.0) + pi * iv_n.fillna(0.0)).where(
            gini_n.notna() | iv_n.notna()
        ),
    }
)
```

```
# =====
# 7. PISO DE RUIDO: ¿EL IV / GINI OBSERVADO SUPERA AL AZAR?
# =====
def piso_ruido_iv(
    n_eventos: int, n_no_eventos: int, n_bins: int, alpha: float = 0.01,
    bonferroni_n: int = 1,
) -> float:
    """IV maximo esperable de una variable de PURO RUIDO, al nivel ``alpha``.
```

Por que hace falta

El IV es una estadistica **estimada**: incluso una variable sin ninguna relacion con el target obtiene un IV estrictamente positivo, porque los bins nunca reparten los eventos en la proporcion exacta de la poblacion. Ese IV espurio CRECE con el numero de bins y DECRECE con el tamano de la muestra. Un umbral fijo (0.02) ignora ambos efectos: es demasiado laxo con pocos datos y muchos bins, y demasiado estricto con muchos datos.

Derivacion

Sean p_i y q_i las distribuciones del bin i entre eventos y no eventos, ambas estimando la misma proporcion π_i bajo la hipotesis nula de independencia. Con $c = 1/n1 + 1/n0$:

$$IV = \sum_i (q_i - p_i) * \ln(q_i / p_i) \approx \sum_i (q_i - p_i)^2 / \pi_i$$

Como $\text{Var}(q_i - p_i) = \pi_i (1 - \pi_i) c$, resulta

$$E[IV \mid H_0] = (k - 1) * c$$

y, mas util todavia, IV / c sigue asintoticamente una chi-cuadrado con $k - 1$ grados de libertad. El piso es entonces el cuantil exacto:

$$\text{piso} = c * \chi^2_{1-\alpha}(k - 1)$$

Interpretacion: un IV por debajo de este valor es indistinguible del que produciria una columna de numeros aleatorios con la misma cantidad de bins

y la misma muestra. No es evidencia de poder predictivo.

Parameters

bonferroni_n

Numero de contrastes simultaneos. Si se evaluan 50 variables, con $\alpha=0.01$ se esperaria media variable espuria por azar; corregir por Bonferroni lo evita, a costa de descartar senales genuinas muy debiles. Por defecto 1 (sin correccion), porque en seleccion de variables suele preferirse el error de conservar de mas al de perder una senal real.

"""

```
from scipy.stats import chi2
```

```
k = int(n_bins)
```

```
if k < 2 or n_eventos < 1 or n_no_eventos < 1:
    return np.nan
```

```
c = 1.0 / n_eventos + 1.0 / n_no_eventos
```

```
alpha_ef = min(max(alpha / max(bonferroni_n, 1), 1e-12), 0.5)
```

```
return float(c * chi2.ppf(1.0 - alpha_ef, df=k - 1))
```

```
def piso_ruido_gini(
```

```
    n_eventos: int, n_no_eventos: int, alpha: float = 0.01, bonferroni_n: int = 1,
```

```
) -> float:
```

```
    """Gini maximo esperable de una variable de puro ruido, al nivel ``alpha``.
```

Bajo la hipotesis nula, el AUC de una variable irrelevante se distribuye alrededor de 0.5 con la varianza de Bamber / Hanley-McNeil:

$$\text{Var}(\text{AUC} \mid H_0) = (n_1 + n_0 + 1) / (12 * n_1 * n_0)$$

Como $\text{Gini} = 2 * \text{AUC} - 1$, se tiene $\text{sd}(\text{Gini}) = 2 * \text{sd}(\text{AUC})$ y el piso es el cuantil normal de una cola:

$$\text{piso} = z_{\{1-\alpha\}} * 2 * \sqrt{(n_1 + n_0 + 1) / (12 * n_1 * n_0)}$$

Advertencia importante: este piso es valido para el Gini calculado sobre los valores CRUDOS. El Gini sobre la proyeccion WOE se estima con un mapeo ajustado sobre los mismos datos, por lo que esta sesgado al alza (usa k-1 parametros libres). Por eso la decision de la fase 2 se apoya en el piso del IV, que si incorpora los grados de libertad del binning.

"""

```
from scipy.stats import norm
```

```
if n_eventos < 1 or n_no_eventos < 1:
    return np.nan
```

```
n1, n0 = float(n_eventos), float(n_no_eventos)
```

```
var_auc = (n1 + n0 + 1.0) / (12.0 * n1 * n0)
```

```
alpha_ef = min(max(alpha / max(bonferroni_n, 1), 1e-12), 0.5)
```

```
return float(norm.ppf(1.0 - alpha_ef) * 2.0 * np.sqrt(var_auc))
```

```
def clasificar_iv(iv: float) -> str:
```

```
    """Traduce un IV a la escala cualitativa estandar de Siddiqi."""
```

```
    if iv is None or not np.isfinite(iv):
```

```
        return "NO_CALCULABLE"
```

```
    if iv < 0.02:
```

```
        return "SIN_PODER"
```

```
    if iv < 0.10:
```

```
        return "DEBIL"
```

```
    if iv < 0.30:
```

```
        return "MEDIO"
```

```
    if iv < 0.50:
```

```
        return "FUERTE"
```

```
    return "SOSPECHOSO_FUGA"
```